

Chapter 8: Pattern Printing in Java

Personal Notes

Pattern problems are one of the best ways to build programming logic in the beginning. They look simple, but they teach loops, row-column thinking, and printing control — all the skills you will later need for matrices, simulation problems, and DSA.

If you are a beginner, do not treat patterns as "just star printing." They build the exact thinking used in nested loops, matrix problems, and most real coding-round questions.

1. Basics You Must Know Before Starting

Before solving any pattern, make sure you understand these five things.

1.1 Outer loop controls rows

If a pattern has n rows, the outer loop runs n times. Each iteration of the outer loop prints one row.

```
for (int row = 1; row <= n; row++) {  
    // each iteration prints one row  
}
```

1.2 Inner loop controls what's inside a row

The inner loop decides how many stars, numbers, or spaces get printed inside the current row.

```
for (int col = 1; col <= n; col++) {  
    System.out.print("* ");  
}
```

1.3 print() vs println()

- `System.out.print()` prints on the same line
- `System.out.println()` moves the cursor to the next line

That is why a pattern looks like this — print inside the inner loop, `println` after it:

```
for (int row = 1; row <= n; row++) {  
    for (int col = 1; col <= n; col++) {  
        System.out.print("* ");  
    }  
    System.out.println();  
}
```

1.4 Always think row by row

The best way to solve a pattern is to look at it one row at a time and ask:

- Are stars increasing or decreasing?
- Are spaces increasing or decreasing?
- Are numbers changing with row or column?
- Is there a formula linking row number to count?

1.5 Most beginner patterns are made of 3 things

- Stars or characters (or numbers / letters)
- Spaces
- Row-column relationship

Once you understand these three, patterns become easy.

2. Pattern 1: Solid Square

Problem

Print a solid square of size n.

Example output for n = 4

```
* * * *
* * * *
* * * *
* * * *
```

Observation

- Total rows = n
- Total columns in every row = n
- Every position prints a star

So:

- outer loop → rows
- inner loop → columns

Java Solution

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
```

```

        for (int row = 1; row <= n; row++) {
            for (int col = 1; col <= n; col++) {
                System.out.print("* ");
            }
            System.out.println();
        }

        sc.close();
    }
}

```

Dry Run for n = 3

Row 1: col = 1 → *, col = 2 → *, col = 3 → *, then line ends

Row 2: again 3 stars

Row 3: again 3 stars

Output:

```

* * *
* * *
* * *

```

Key Learning

This is the most basic pattern. It teaches:

- fixed rows
- fixed columns
- nested loop structure

3. Pattern 2: Hollow Square

Problem

Print a hollow square of size n.

Example output for n = 5

```

* * * * *
*       *
*       *
*       *
* * * * *

```

Observation

In a hollow square:

- first row → all stars
- last row → all stars
- first column → star
- last column → star
- everything else → space

So a star is printed when row == 1, or row == n, or col == 1, or col == n.

Java Solution

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        for (int row = 1; row <= n; row++) {
            for (int col = 1; col <= n; col++) {
                if (row == 1 || row == n || col == 1 || col == n) {
                    System.out.print("* ");
                } else {
                    System.out.print("  ");
                }
            }
            System.out.println();
        }

        sc.close();
    }
}
```

Dry Run for n = 4

Row 1 → row == 1, so all stars: * * * *

Row 2 → col 1 = star, col 2 = space, col 3 = space, col 4 = star: * *

Row 3 → same as row 2

Row 4 → row == n, so all stars: * * * *

```
* * * *
*      *
*      *
* * * *
```

Key Learning

This pattern teaches boundary checking. Beginner logic improves a lot when you start thinking in terms of:

- top boundary
- bottom boundary
- left boundary
- right boundary

4. Pattern 3: Solid Right-Angle Triangle

Problem

Print a right-angle triangle of size n.

Example output for n = 5

```
*
* *
* * *
* * * *
* * * * *
```

Observation

- row 1 → 1 star
- row 2 → 2 stars
- row 3 → 3 stars

So in every row, number of stars = row number.

Java Solution

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        for (int row = 1; row <= n; row++) {
            for (int col = 1; col <= row; col++) {
                System.out.print("* ");
            }
            System.out.println();
        }

        sc.close();
    }
}
```

Dry Run for n = 4

Row 1 → 1 star, Row 2 → 2 stars, Row 3 → 3 stars, Row 4 → 4 stars.

```
*  
* *  
* * *  
* * * *
```

Key Learning

This pattern teaches increasing inner loop length. This idea is the base for:

- increasing stars
- decreasing stars
- increasing spaces

5. Pattern 4: Inverted Right-Angle Triangle

Problem

Print an inverted right-angle triangle of size n.

Example output for n = 5

```
* * * * *  
* * * *  
* * *  
* *  
*
```

Observation

- row 1 → 5 stars
- row 2 → 4 stars
- row 3 → 3 stars

Stars are decreasing. The formula for stars in each row is:

```
stars = n - row + 1
```

Java Solution

```
import java.util.Scanner;  
  
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int n = sc.nextInt();  
  
        for (int row = 1; row <= n; row++) {
```

```

        for (int col = 1; col <= n - row + 1; col++) {
            System.out.print("* ");
        }
        System.out.println();
    }

    sc.close();
}

```

Dry Run for n = 4

Row 1 → $4 - 1 + 1 = 4$ stars

Row 2 → $4 - 2 + 1 = 3$ stars

Row 3 → $4 - 3 + 1 = 2$ stars

Row 4 → $4 - 4 + 1 = 1$ star

```

* * * *
* * *
* *
*

```

Key Learning

This pattern teaches decreasing loop count. Whenever something shrinks row by row, think of the formula $n - \text{row} + 1$.

6. Pattern 5: Solid Pyramid

Problem

Print a solid pyramid of height n.

Example output for n = 4

```

    *
   * * *
  * * * * *
 * * * * * *

```

Observation

Each row has two parts: leading spaces and stars.

Spaces

- row 1 → 3 leading spaces
- row 2 → 2 leading spaces
- row 3 → 1 leading space

- row 4 → 0 leading spaces

So spaces = $n - \text{row}$.

Stars

- row 1 → 1 star
- row 2 → 3 stars
- row 3 → 5 stars
- row 4 → 7 stars

So stars = $2 * \text{row} - 1$.

Java Solution

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        for (int row = 1; row <= n; row++) {

            for (int space = 1; space <= n - row; space++) {
                System.out.print(" ");
            }

            for (int star = 1; star <= 2 * row - 1; star++) {
                System.out.print("* ");
            }

            System.out.println();
        }

        sc.close();
    }
}
```

Dry Run for n = 3

Row 1 → spaces = 2, stars = 1

Row 2 → spaces = 1, stars = 3

Row 3 → spaces = 0, stars = 5

```

    *
  * * *
* * * * *
```


Key Learning

This pattern teaches combining spaces, stars, and formula-based printing. It is the base for:

- diamond
- hollow pyramid
- butterfly
- hourglass
- rhombus

7. How to Approach Any Pattern Problem

Whenever you see a new pattern, follow this 4-step method.

Step 1 — Count rows

How many rows does the pattern have? That decides the outer loop.

Step 2 — For each row, count parts

For every row, count:

- number of spaces
- number of stars / numbers / alphabets

Step 3 — Find the formula

Look at how counts change from row to row:

- Are they increasing?
- Decreasing?
- Odd count? Even count?
- Linked to row number, like $2 * \text{row} - 1$?

Step 4 — Write separate loops for each part

One loop for spaces, one loop for stars, one for closing spaces if needed. Keep them separate inside the outer loop.

Tip: Most patterns become easy once you stop seeing the whole design and start seeing it row by row.

8. Quick Reference: Formulas to Remember

Pattern	Shape Idea	Key Formula
Solid square	Fixed $n \times n$ grid	stars = n in every row

Pattern	Shape Idea	Key Formula
Hollow square	Border only	star if row/col is 1 or n
Right-angle triangle	Stars grow each row	stars = row
Inverted triangle	Stars shrink each row	stars = n - row + 1
Solid pyramid	Centered pyramid	spaces = n - row, stars = $2 * \text{row} - 1$
Inverted pyramid	Pyramid upside down	spaces = row - 1, stars = $2 * (n - \text{row}) + 1$
Diamond	Pyramid + inverted	combine both formulas
Butterfly	Stars on both sides	stars = row, spaces = $2 * (n - \text{row})$

9. Assignment Patterns (Practice on Your Own)

Now practice these on your own. The shape for each one is given so you know exactly what to print. Use the 4-step approach from Section 7.

1. Solid Rectangle — for rows = 3, cols = 5

```
* * * * *
* * * * *
* * * * *
```

2. Hollow Rectangle — for rows = 4, cols = 6

```
* * * * * *
*           *
*           *
* * * * * *
```

3. Hollow Right-Angle Triangle — for n = 5

```
*
* *
*  *
*   *
* * * * *
```

4. Inverted Solid Pyramid — for n = 4

```
* * * * * * *
* * * * *
* * *
* * *
```

```
*

```

5. Hollow Pyramid — for n = 5

```
      *
     * *
    *   *
   *     *
  *       *
 *         *
* * * * *

```

6. Solid Diamond — for n = 4

```
      *
     * * *
    * * * * *
   * * * * * *
  * * * * * * *
 * * * * *
  * * *
   *

```

7. Hollow Diamond — for n = 4

```
      *
     *   *
    *     *
   *       *
  *         *
 *           *
*             *
 *           *
  *         *
   *       *
    *     *
     *   *
      *

```

8. Butterfly — for n = 4

```
*               *
* *           * *
* * *       * * *
* * * * *   * * *
* * * * * * * *
* * * * * * * *
* * *       * * *
* *           * *
*               *

```

9. Rhombus — for n = 5

```
      * * * * *
     * * * * *
    * * * * *
   * * * * *
  * * * * *
 * * * * *
* * * * *

```

10. Number Triangle — for n = 5

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

11. Symmetric Number Pyramid — for n = 4

```
      1
     1 2 1
    1 2 3 2 1
   1 2 3 4 3 2 1
```

12. Number Pyramid — for n = 4

```
      1
     2 2
    3 3 3
   4 4 4 4
```

13. Floyd's Triangle — for n = 5

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

14. Alphabet Triangle — for n = 5

```
A
A B
A B C
A B C D
A B C D E
```

15. Inverted Alphabet Triangle — for n = 5

```
A B C D E
A B C D
A B C
A B
A
```

16. Symmetric Alphabet Pyramid — for n = 4

```
      A
     A B A
    A B C B A
```

```
A B C D C B A
```

17. Reverse Alphabet Right-Angle Triangle — for n = 5

```
E
E D
E D C
E D C B
E D C B A
```

18. Pascal's Triangle — for n = 5

```
      1
     1 1
    1 2 1
   1 3 3 1
  1 4 6 4 1
```

19. Hourglass Shape — for n = 4

```
* * * * *
  * * * *
   * * *
    *
   * * *
  * * * * *
 * * * * * * *
```

20. Zig-Zag — for n = 9

```
  *      *
*   *   *   *
    *       *
```

21. Spiral Matrix — for n = 4

```
1  2  3  4
12 13 14 5
11 16 15 6
10 9  8  7
```

10. Practice Advice for Beginners

Do not jump straight to hard patterns. Follow this order so confidence builds in the right direction:

1. Solid square
2. Hollow square
3. Solid rectangle
4. Hollow rectangle

5. Right-angle triangle
6. Inverted triangle
7. Pyramid
8. Diamond
9. Butterfly
10. Number and alphabet patterns
11. Pascal's triangle, spiral matrix, zig-zag

11. Final Advice

If you get stuck on any pattern, do not panic. Just take a piece of paper and write down three things for each row:

- the row number
- how many spaces in that row
- how many symbols in that row

That single habit solves almost every beginner pattern question.

Key Takeaway: Patterns are not about creativity — they are about observation. Count rows, count parts, find the formula, write separate loops. Master this and nested loops, matrices, and DSA simulation problems will feel much easier.